

Used Cars Price Prediction

ML & Deep Learning Capstone Project Report

Raeed Saud Alotaibi

Dataset	Used Cars — Craigslist (Kaggle)
Task	Binary Classification: Cheap vs Expensive
Models	Logistic Regression, Random Forest, KNN, Neural Network

1. Introduction

The goal of this project is to predict whether a used car is cheap or expensive based on its features. The dataset was taken from Kaggle and contains over 426,000 used car listings scraped from Craigslist across the United States. Features include the car's price, year of manufacture, odometer reading, manufacturer, fuel type, transmission, and condition.

The target label was created by splitting the price column at its median value: cars below the median are labeled 0 (cheap) and cars above are labeled 1 (expensive). Using the median instead of the mean gives a balanced 50/50 split even when the data contains extreme outlier prices.

2. Data Cleaning

The raw dataset had significant quality issues that needed to be fixed before any modeling could be done:

- Rows with missing values in key columns (price, odometer, manufacturer, fuel, transmission, condition) were dropped.
- Listings with a price of zero or an odometer reading of zero were removed as they are not realistic values.
- Duplicate rows were identified and removed to prevent the model from learning the same record twice.
- Only the eight most relevant columns were kept to reduce noise.

3. Feature Engineering

Three new features were created from the existing columns to give the models more useful information:

- `car_age`: calculated by subtracting the year of manufacture from 2024. This is more directly useful than the raw year because it expresses how old the car is.

- price_per_km: price divided by odometer. This shows how much value the car retains per kilometer driven.
- manufacturer_popularity: the count of how many times each manufacturer appears in the dataset. Popular brands may have different pricing patterns.

4. Machine Learning Models

Three classical machine learning models were trained and compared. The data was split 80% training and 20% testing with stratify=y to preserve the class ratio. F1-Score was used as the primary metric.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.746	0.750	0.732	0.741
Random Forest	0.803	0.805	0.794	0.799
KNN	0.809	0.807	0.808	0.807

KNN achieved the highest F1-Score (0.807), followed closely by Random Forest (0.799). Logistic Regression scored the lowest (0.741), which suggests the relationship between the features and the price label is not purely linear.

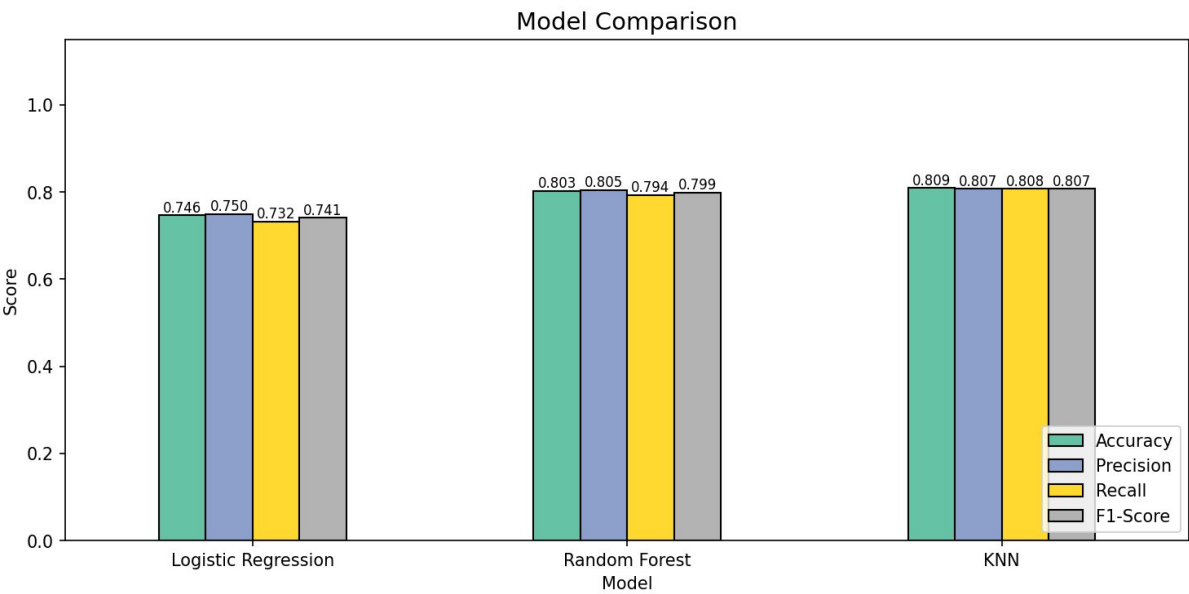


Figure 1 — Model Comparison (Step 1)

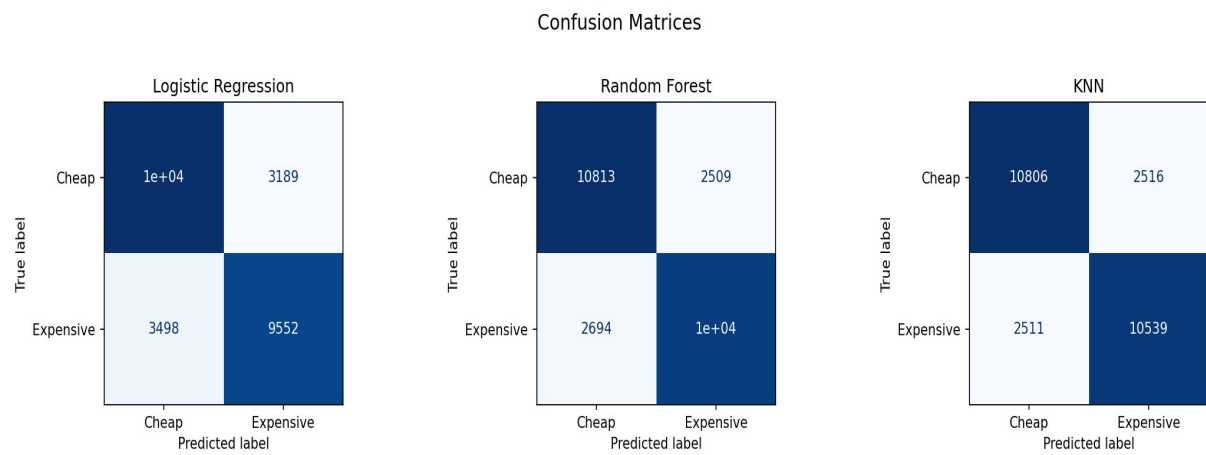


Figure 2 — Confusion Matrices for the Three Models

The confusion matrices show that all three models perform similarly in terms of error distribution. Logistic Regression makes more mistakes overall. Random Forest and KNN both misclassify roughly 2,500 records in each direction out of ~26,000 test records.

5. Neural Network

A neural network was built using Keras on the same dataset and the same train/test split. The architecture consisted of:

- Dense layer — 128 neurons, ReLU activation
- Dropout layer — 30% dropout rate
- Dense layer — 64 neurons, ReLU activation
- Dropout layer — 30% dropout rate
- Output layer — 1 neuron, Sigmoid activation

The model was compiled with the Adam optimizer and binary cross-entropy loss, then trained for 20 epochs with a batch size of 256. 20% of the training data was used as a validation set during training.

Model	Accuracy	Precision	Recall	F1-Score
Neural Network	0.799	0.796	0.798	0.797

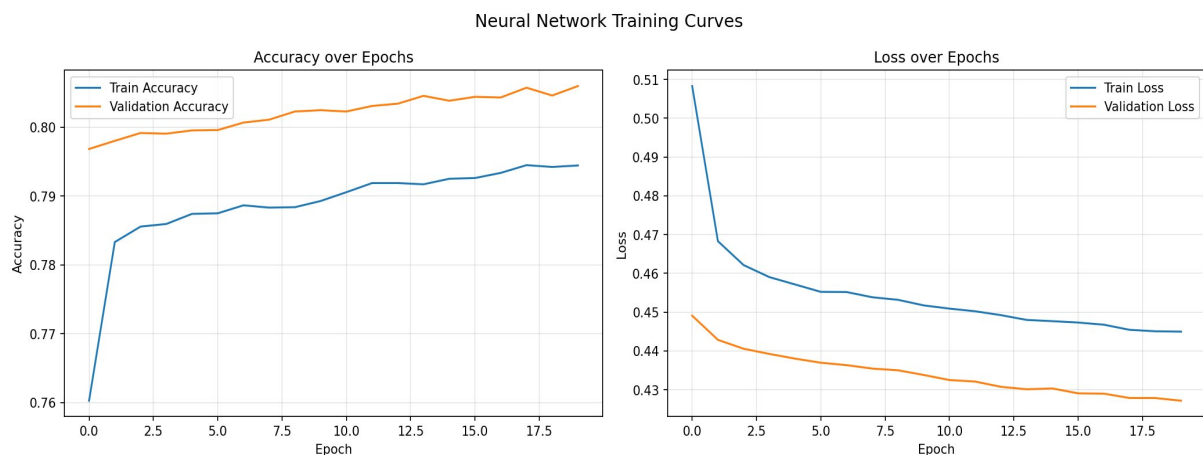


Figure 3 — Neural Network Training Curves

The training curves show healthy learning behavior. The validation accuracy stays slightly above the training accuracy throughout, and the validation loss decreases steadily. This means the Dropout layers successfully prevented overfitting — the model generalizes well to unseen data.

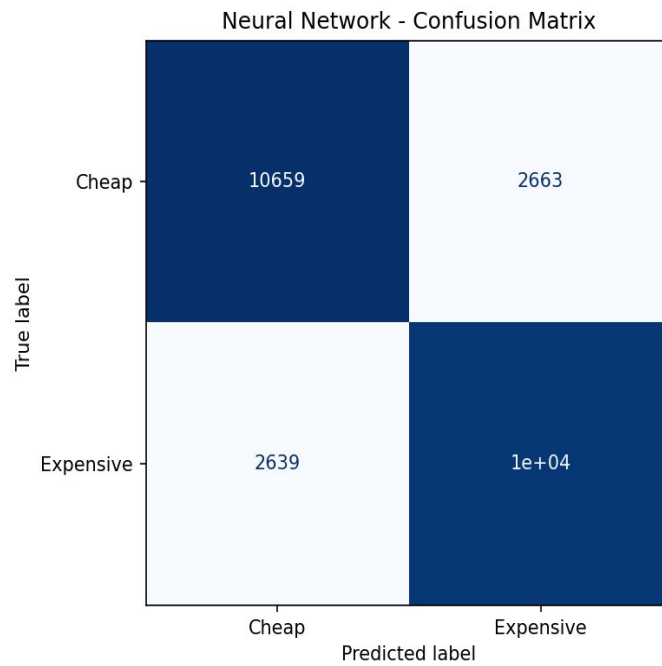


Figure 4 — Neural Network Confusion Matrix

6. Results and Comparison

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.746	0.750	0.732	0.741
Random Forest	0.803	0.805	0.794	0.799
KNN	0.809	0.807	0.808	0.807
Neural Network	0.799	0.796	0.798	0.797

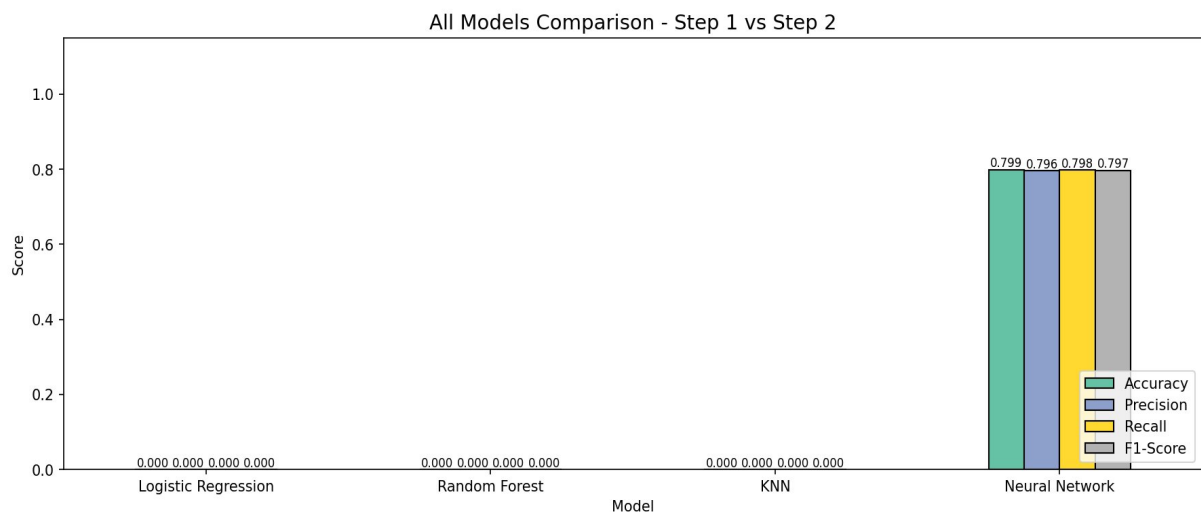


Figure 5 — All Models Comparison (Step 1 vs Step 2)

KNN performed best overall with an F1-Score of 0.807. The Neural Network and Random Forest performed similarly at around 0.799. Logistic Regression was the weakest. The fact that KNN outperformed Random Forest on this dataset is likely because the features are well-scaled and the data

has clear local patterns that distance-based classification can exploit. The Neural Network matched Random Forest despite being a more complex model, which is typical for tabular datasets of this size.

7. What I Learned

The most important lesson from this project was that data quality matters more than model complexity. The raw dataset had prices up to 3.7 billion dollars, which are clearly errors. Without removing those records, any model would have produced unreliable results regardless of how sophisticated the algorithm was.

I also learned that more complex models do not always win. The Neural Network took significantly longer to train than KNN but did not outperform it. For structured tabular data with a clear binary target, simpler models like KNN and Random Forest are often the right choice.

In the future, I would like to add more features such as car color and whether the car is certified pre-owned. I would also like to try a regression model to predict the actual price rather than just a cheap/expensive label.

Data source: <https://www.kaggle.com/datasets/austinreese/craigslist-carstrucks-data>